

Proje 4: E-Ticaret Uygulaması (Otomatik Ölçeklendirme ve Yönetim)

YouTube: <https://youtu.be/AA6HyNWqcXU>

Bu rapor, Java Spring Boot tabanlı bir e-ticaret uygulamasının AWS üzerinde yüksek erişilebilirlik, yük dengeleme ve otomatik ölçeklendirme prensiplerine uygun olarak yapılandırılması sürecini ve teknik detaylarını özetlemektedir.

1. Proje Kapsamı ve Hedefler

Projenin temel amacı, değişen trafik yoğunluklarına dinamik olarak yanıt verebilen, kesintisiz çalışan ve veritabanı entegrasyonu tamamlanmış modüler bir bulut mimarisi oluşturmaktır.

Belirlenen gereksinimler doğrultusunda sistem; tek bir sunucu arızasından etkilenmeyecek şekilde mimari edilmiştir.

2. Teknolojik Altyapı ve Bileşenler

Sistem genelinde kullanılan backend, veritabanı ve bulut teknolojileri aşağıdaki tabloda detaylandırılmıştır:

Backend Dili & Framework: Java 17 / Spring Boot

Veritabanı: MySQL

Sanal Sunucu Altyapısı: AWS EC2

Yük Dengeleyici: AWS Elastic Load Balancer

Ölçeklendirme Grubu: AWS Auto Scaling Group

3. Adım Adım Kurulum ve Yapılandırma Aşamaları

3.1. Temel Sunucu ve Alt Yapı İmajının (AMI) Hazırlanması

İlk aşamada, uygulamanın çalışacağı kararlı bir ana imaj elde etmek için şu adımlar izlenmiştir:

1. Ubuntu işletim sistemine sahip bir adet temel EC2 sunucusu ayağa kaldırılmıştır.
2. Sunucu içerisine projenin bağımlılığı olan OpenJDK 17 kurulumu gerçekleştirilmiştir.
3. Derlenen Spring Boot uygulaması (.jar dosyası) güvenli yollarla sunucuya aktarılmıştır.
4. Uygulama manuel olarak test edildikten ve veritabanı bağlantısı doğrulandıktan sonra, sunucunun o anki durumunu sabitleyen eticaret-ami-v2 isimli özel bir Amazon Machine Image (AMI) oluşturulmuştur.

3.2. Başlatma Şablonu (Launch Template) Yapılandırması

Auto Scaling sisteminin yeni sunucuları otomatik olarak hangi ayarlarla başlatacağını tanımlamak için eticaret-template-v2 oluşturulmuştur:

- İmaj: Hazırlanan eticaret-ami-v2 seçilmiştir.
- Sunucu Tipi: Maliyet ve performans optimizasyonu için t3.micro tercih edilmiştir.
- Ağ Ayarları: Auto-assign public IP özelliği aktif edilmiştir.
- Kullanıcı Verisi: Sunucular ilk açıldığında uygulamanın otomatik olarak başlamasını sağlayan aşağıdaki Bash betiği eklenmiştir:

```
#!/bin/bash cd /home/Ubuntu nohup java -jar eticaret-0.0.1-SNAPSHOT.jar &
```

3.3. Hedef Grup (Target Group) ve Yük Dengeleyici (ALB) Kurulumu

Gelen ağ trafiğini sunuculara dağıtmak amacıyla yönlendirme mekanizması şu şekilde kurulmuştur:

1. eticaret-tg-v2 adında Instance tabanlı bir Hedef Grup tanımlanmıştır. Çalışma portu 8080 olarak belirlenmiştir.
2. Sistemin sunucu sağlığını kontrol edebilmesi için Sağlık Kontrolü Yolu (Health Check Path) /api/products olarak yapılandırılmıştır.
3. eticaret-alb-v2 isimli Application Load Balancer oluşturulmuş, dış dünyadan gelecek HTTP 80 isteklerini dinleyecek şekilde konfigüre edilerek doğrudan bu Hedef Gruba yönlendirilmiştir.

3.4. Auto Scaling Grubu Entegrasyonu

Yüksek erişilebilirlik hedefine ulaşmak için oluşturulan Auto Scaling grubu, hazırlanan Başlatma Şablonunu ve Hedef Grubu birbirine bağlamıştır. Kapasite sınırları minimum 2 sunucu her an aktif olacak, yoğun anlarda ise 4 sunucuya kadar genişleyebilecek şekilde set edilmiştir.

4. Karşılaşılan Sorunlar ve Uygulanan Çözüm Stratejileri

4.1. Güvenlik Grubu Port Blokajları (Timed Out)

Sorun: Sunucuya dışarıdan erişilmeye çalışıldığında zaman aşımı hatası alınmıştır.

Çözüm: AWS Güvenlik Grubu kuralları incelenmiş; uygulamanın dinlediği 8080 portunun ve yönetimsel erişim için 22 (SSH) portunun dış dünyaya (0.0.0.0/0) açık olmadığı fark edilerek gerekli gelen kuralları tanımlanmıştır.

4.2. HTTP/HTTPS Karışıklığı ve 502 Bad Gateway

Sorun: Yük dengeleyici DNS adresi üzerinden yapılan çağrılarda 502 Bad Gateway veya "Error parsing HTTP request header" hatalarıyla karşılaşmıştır.

Çözüm: Modern tarayıcıların adresi otomatik olarak HTTPS protokolüne zorladığı, ancak mevcut mimarinin uçtan uca düz HTTP (Port 80-> Port 8080) mimarisine göre tasarlandığı tespit edilmiştir. İsteklerin başına açıkça http:// protokolü eklenerek ve adres sonundaki gereksiz :8080 port ibaresi kaldırılarak entegrasyonun sorunsuz çalışması sağlanmıştır.

4.3. SSH Anahtar Kaybı Yönetimi

Sorun: Sunucu loglarına müdahale edilmesi gerektiğinde yerel `.pem` anahtar dosyasının kaybolduğu fark edilmiştir.

Çözüm: Fiziksel anahtar bağımlılığını ortadan kaldıran AWS tarayıcı tabanlı güvenli bağlantı mimarisi (EC2 Instance Connect) kullanılarak sunucu terminaline doğrudan erişim sağlanmış ve log analizleri bu şekilde yürütülmüştür.

5. Sonuç ve Sistem Durumu

Yapılan tüm optimizasyonların ardından Spring Boot uygulamasının başarıyla ayağa kalktığı, veritabanı havuzunu kurduğu ve yönlendirici sağlık testlerine olumlu yanıt verdiği doğrulanmıştır. Sistem, otomatik ölçeklendirme altyapısı sayesinde kesintisiz ve yüksek performanslı bir şekilde canlı operasyona hazır durumdadır.

```
1 • ⊖ INSERT INTO eticaret.products VALUES (  
2     3,  
3     "AWS Test Ürünü 3",  
4     300,  
5     30 );  
6
```

```
{"id":3,"name":"AWS Test Ürünü 3","price":300.0,"stock":30}]
```